

FlyRank Backend Track — Assignment A4

Auth: Login & Protect — Supabase Auth, JWT Verification, & Swagger UI

Author: Deepak	Track: Backend & Web Security	Status: Completed & Pushed
GitHub: github.com/deepak007679/supabase-auth-api	Commits: 9 Sequential Commits	IdP: Supabase Auth

1. Executive Summary & The Auth Trust Triangle

Modern backend architectures never store plaintext passwords or roll homebrewed cryptography. Instead, the backend relies on an established **Identity Provider (IdP)** — Supabase. Authentication operates as a three-party trust triangle: the client authenticates against Supabase to acquire a cryptographically signed JWT; subsequent API calls send this token in the Authorization: Bearer <token> header; and our server's reusable middleware guard verifies the token with Supabase before granting access.

2. Implementation Stages & Verified Commit History

Stage	Commit Message	Key Deliverables & Actions Taken	Status
Stage 0	Stage 0: setup server and supabase client	Set up .gitignore, committed .env.example, initialized @supabase/supabase-js client with offline fallback.	PASSED
Stage 1	Stage 1: signup and login routes working	Implemented POST /auth/signup (201 Created) and POST /auth/login (200 with JWT access & refresh tokens).	PASSED
Stage 2	Stage 2: public route and unverified protected route	Created open GET /public/info (200) and unverified GET /protected/profile requiring Bearer header.	PASSED
Stage 3	Stage 3: profile route token verification	Integrated supabase.auth.getUser(token) on /protected/profile; verified forged token returns 401.	PASSED
Stage 4	Stage 4: auth middleware and logout endpoint	Extracted requireAuth middleware; added POST /auth/logout (204) and GET /protected/dashboard.	PASSED
Stage 5	Stage 5: Swagger UI documentation with bearer auth	Configured openapi.json with BearerAuth security scheme; served Swagger UI at /docs; generated screenshot.	PASSED
Stage 6	Stage 6: publish to GitHub and write README	Created public GitHub repository, published code with comprehensive README and curl examples.	PASSED
Stage 7	Stage 7: AI vs me	Quarantined AI code in ai-version/; compared diffs; wrote comprehensive review answering 3 security questions.	PASSED
Extras	Extras: role-based 403, token refresh, and login rate limiting	Added /protected/admin (403 Forbidden), /auth/refresh token rotation, and 429 login brute-force limiter.	PASSED

3. Technical Architecture & Security Highlights

A. Reusable Auth Middleware Guard (authMiddleware.js)

Rather than repeating authentication logic in every route handler, token extraction and Supabase verification are consolidated into a single middleware function. It validates the Bearer prefix, checks for non-empty token payload, calls `supabase.auth.getUser(token)`, and attaches `req.user`.

```
async function requireAuth(req, res, next) {
  const authHeader = req.headers['authorization'];
  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'Access token required' });
  }
  const token = authHeader.split(' ')[1];
  try {
    const { data, error } = await supabase.auth.getUser(token);
    if (error || !data?.user) return res.status(401).json({ error: 'Invalid or expired token' });
    req.user = data.user;
    next();
  } catch (err) {
    return res.status(401).json({ error: 'Invalid or expired token' });
  }
}
```

B. The 401 Unauthorized vs 403 Forbidden Paradigm

A critical distinction in web security is differentiating between **Authentication** and **Authorization**:

- **401 Unauthorized**: *'I do not know who you are.'* Occurs when the token is missing, expired, or tampered with.
- **403 Forbidden**: *'I know who you are, but you are not allowed in.'* Occurs when an authenticated user attempts to access a restricted endpoint (e.g. `GET /protected/admin`) without the required administrative privileges.

C. Stage 7 Code Review: The AI Rematch

When comparing the hand-built implementation with the quarantined AI solution in `ai-version/`, three major architectural differences were uncovered:

1. **Naive Header Extraction**: The AI used `auth.replace("Bearer ", "")`, allowing malformed headers without the 'Bearer' scheme to slip through unvalidated.
2. **Unhanded Exceptions**: The AI neglected `try...catch` wrapping around `getUser`, leaving the server vulnerable to unhandled promise rejections during network outages.
3. **Omission of Swagger UI**: The AI completely skipped Swagger / OpenAPI setup because the prompt did not prescribe exact schema requirements.

4. Submission Artifacts

• **GitHub Repository**: <https://github.com/deepak007679/supabase-auth-api> • **Swagger Documentation Screenshot**: `screenshots/swagger-auth.png` • **Environment Templates**: `.env.example` (committed), `.env` (git-ignored) • **Automated Test Suite**: `test_auth.js` (12 passed checkpoints) • **Quarantined AI Version**: `ai-version/server.js` & `ai-version/prompt.txt`